

本地模拟器详解

与附件 1/2 协议一致的无线电干扰源仿真环境
——物理模型、时间记账、协议层、误差模型与验证

2026 高教社杯全国大学生数学建模竞赛 B 题

2026 年 9 月 11 日

摘要

本文档完整说明为解决本题而实现的本地模拟器（code/simulator.py，416 行）。它由三层构成：**物理内核 Arena**（场地、干扰源、运动与虚拟时间记账）、**HTTP 协议层**（复现附件 2 的四条指令、字段校验与幂等语义）、**算例生成器**（由随机种子唯一确定一局测试）。

文档给出：时间记账的完整规则与边界情形、 $\pm 1^\circ$ 示向度误差的**地点确定性**实现、定向源覆盖判据、协议校验流水线与状态码矩阵、日志格式规范、与附件 2 的逐条对照表、以及**已知差异与接入官方模拟器的方法**。

诚实披露：官方模拟器需通过百度网盘分发并用参赛队号联网注册，本次环境**未能获取**，因此论文与报告中的”正式测试”结果来自本模拟器；接口与时间规则严格按附件 2 复现，但不属于官方成绩。

目录

1	设计目标	2
2	架构	2
3	物理模型	3
3.1	常量	3
3.2	干扰源 Source	3
3.3	算例生成	3
4	时间记账	4
5	示向度误差模型	5
5.1	题面要求	5
5.2	实现：地点哈希	5
5.3	数值检查	5
6	HTTP 协议层	5
6.1	校验流水线	5

6.2 幂等语义	6
6.3 字段规范	6
7 日志规范	7
8 三种运行模式	7
9 与附件协议的逐条对照	8
10 已知差异与局限	8
10.1 如何接入官方模拟器	9

1 设计目标

1. **协议忠实**：机器狗程序必须真实走 **HTTP** (`POST /enter /measure /clear /exit`)，而不是调用内存函数——这样策略代码在换到官方模拟器时不需要任何改动。
2. **时间忠实**：虚拟时间的记账规则必须与附件 1 完全一致，否则”平均定位清除时间”这一指标失真，全部优化失去意义。
3. **确定性**：同一算例、同一策略必须得到完全相同的结果。随机性只来自显式种子；示向度误差必须与”同一地点重复测量结果相同”这一题设吻合。
4. **可观测**：每个动作落一行 JSON 日志，便于事后逐步复核与统计。
5. **可注入**：既能作为阻塞服务运行（正式测试），也能作为后台线程运行（自动化脚本），还能以纯内存客户端调用（大规模调参）。

2 架构

```

1 +-----+
2 | 算例生成器 make_case(rng, n_sources, kind_mix, ...) |
3 |   -> [Source(channel, x, y, r_eff, kind, direction), ...] |
4 +-----+
5 | HTTP 协议层 _Handler / ArenaHTTPServer / serve()   |
6 |   路由 -> 内容类型 -> 体积 -> JSON -> 字段白名单 -> |
7 |   必填字段 -> arena_id/robot_id -> position/channel 校验 |
8 |   -> 幂等缓存 -> 调用 Arena -> 回填 real_timestamp_ms |
9 +-----+
10 | 物理内核 Arena                                     |
11 |   enter / exit / measure / clear                   |
12 |   虚拟时间记账、探测判据、清除判定、日志落盘、统计 |
13 +-----+

```

代码 1 三层结构与职责边界

分层的意义：物理内核不知道 HTTP 的存在，协议层不知道几何的存在。因此物理内核可以被 `LocalClient` 直接调用（快一个数量级），而协议层可以脱离策略单独用 `curl` 测试。

3 物理模型

3.1 常量

常量	值	单位	依据
ARENA_RADIUS	1800	m	题面目标区域半径
SPEED	5	m/s	题面移动速度
MEASURE_TIME	5	s	一次检测耗时
SWITCH_TIME	1	s	任意两频道切换耗时
CLEAR_TIME_HIT	5	s	命中：光学 3 s + 激光 2 s
CLEAR_TIME_MISS	3	s	未命中：仅光学
NEAR_RADIUS	5	m	距离 $\leq$ 此值返回 <code>near</code>
CLEAR_RADIUS	20	m	清除成功半径，与角度无关
R_EFF_LO/HI	1000 / 1500	m	每源随机取值
BEARING_ERROR	1.0	°	误差绝对值上界
MAX_VIRTUAL_DURATION	360000	s	虚拟世界活动时长上限（100 h）
MAX_REAL_DURATION	1200	s	程序运行时间上限（20 min）

3.2 干扰源 Source

```

1 class Source:
2     def __init__(self, channel, x, y, r_eff, kind="omni", direction=None)
3     def pos(self)                # (x, y)
4
5     def covers(self, px, py):
6         if self.kind == "omni":
7             return True
8         vx, vy = px - self.x, py - self.y
9         a = math.radians(self.direction)
10        return (vx*math.cos(a) + vy*math.sin(a)) >= 0.0 # 180 度半平面

```

代码 2 干扰源的两种覆盖判据

探测成立需要同时满足两个条件（`Arena.measure` 中）： $|p - q| \leq R_{\text{eff}}$ 且 `src.covers(p)`。注意：

- `covers` 用 ≥ 0 （含边界），与题面“含边界”一致；
- 清除判定只用距离（ ≤ 20 m），不调用 `covers` ——对应题面“光学精确定位及清除操作是否成功仅与清除点与干扰源之间的距离有关”。

3.3 算例生成

```

1 def make_case(rng, n_sources=None, n_channels=20, kind_mix=0.0,
2               r_eff_lo=1000, r_eff_hi=1500, radius=1800, min_sep=0.0):
3     if n_sources is None:
4         n_sources = rng.randint(10, 16) # 题面：10~16 个
5         chans = rng.sample(range(1, 21), n_sources) # 频道互异
6         for each source:
7             a = 2*pi*rng.random(); r = radius*sqrt(rng.random()) # 面积均匀采样

```

```

8     kind = "directional" if rng.random() < kind_mix else "omni"
9     direc = rng.uniform(0, 360) if kind == "directional" else None
10    r_eff = rng.uniform(r_eff_lo, r_eff_hi)

```

代码 3 make_case: 由种子唯一确定一局

两个容易写错的细节:

1. 半径必须用 $R\sqrt{U}$ 而不是 $R \cdot U$, 否则点在圆心上过密 (不是面积均匀);
2. 频道号从 $\{1, \dots, 20\}$ 中无放回抽样, 天然保证互不相同。

4 时间记账

这是整个模拟器最容易写错、也最影响结论的部分。逐条列出规则:

动作	虚拟时间增量	边界情形
/enter	0	重复 /enter 返回 accepted=false, 不推进时间
/exit	0	未 /enter 直接 /exit 同样拒绝
/measure	$d/5 + [\text{切换}] + 5$	切换项仅当目标频道 $\neq$ 当前频道; 无论结果如何都计满 5 s
/clear	$d/5 + (5 \text{ 或 } 3)$	命中 5 s、未命中 3 s; 清除不改变当前频道 (题面明确)
——	——	d 由前后两次 position 的直线距离算出 (移动一律按直线计)

三条易错点

- (1) 切换时间只算一次: 移动到新位置后再测同频道, 不产生切换耗时; 但如果是”测 ch1 $\rightarrow$ 测 ch2 $\rightarrow$ 再测 ch1”, 第二次要再算 1 s。代码里用 `if channel != self.channel: switch = SWITCH_TIME` 处理, 并把当前频道更新为目标频道。
- (2) 清除不改频道: 这是题面的明确约定, 若实现成”清除后频道变为该频道”, 会系统性低估后续测量的切换时间。
- (3) 检测一律 5 s: 包括 no_signal 与 near, 不存在”快速探测”。

```

1 moved, move_t = self._step(target)    # 移动到目标点, 累加 travel 与 virtual_time
2 switch = SWITCH_TIME if channel != self.channel else 0.0
3 self.channel = channel
4 self.virtual_time += switch + MEASURE_TIME
5 src = self.by_channel.get(channel)
6 result = "no_signal"
7 if src is not None and not src.cleared:
8     dist = hypot(target - src.pos)
9     if dist <= src.r_eff and src.covers(*target):
10         if dist <= NEAR_RADIUS: result = "near"
11         else:
12             result = "direction"
13             svd = (true_bearing + _loc_error(*target, channel)) % 360

```

代码 4 Arena.measure 的记账与判据 (节选)

5 示向度误差模型

5.1 题面要求

附件 1 规定：误差绝对值不超过 1° ，且“同一地点重复测量结果相同”。后者是硬约束——如果实现成“每次测量重新随机”，那么“在某点重测取平均”这种策略会虚假地变好，整个信息获取模型都会失真。

5.2 实现：地点哈希

```
1 def _loc_error(px, py, channel):
2     key = "%.2f|%.2f|%d" % (round(px, 2), round(py, 2), channel)
3     h = zlib.crc32(key.encode("utf-8")) & 0xFFFFFFFF
4     return -1.0 + 2.0 * (h / 4294967295.0) # [-1, 1] 度
```

代码 5 地点确定的误差（crc32 哈希）

三条设计考虑：

1. **确定性：**误差是（位置, 频道）的纯函数，不依赖随机数发生器状态，因此无论机器人以什么顺序、在什么时刻到达该点，读数完全一致；
2. **与地点连续相关（弱）：**由于按 0.01 m 取整，微小移动会跳到完全不同的误差值，这正是“误差是地点函数但无空间结构”的最保守假设——不给出任何可利用的平滑性，策略不能靠“附近误差相近”取巧；
3. **为什么按频道区分：**同一点对两个不同频道的误差相互独立，符合“不同源信号互不影响”。

5.3 数值检查

性质	期望	实测
误差范围	$[-1^\circ, 1^\circ]$	$[-0.9999, 0.9999]^\circ$ (10^5 次抽样)
均值	≈ 0	-3×10^{-4}
同点重测一致	完全相同	完全相同（逐位相等）
相邻 0.01 m 点相关	无相关	无相关（这正是设计目标）

6 HTTP 协议层

6.1 校验流水线

```
1 path in {/enter,/measure,/clear,/exit} else 404
2 Content-Type startswith application/json else 415
3 Content-Encoding in {"", "identity"} else 415
4 Content-Length <= 65536 else 413
5 body 是合法 UTF-8 JSON 对象 else 400
6 字段白名单（多出的字段） -> 200 accepted=false
7 必填 arena_id / robot_id / request_id else 400
8 arena_id == "default" -> 200 accepted=false
9 robot_id 与首次 enter 的一致（若已绑定） -> 200 accepted=false
```

```
10 position 是 {x,y} 且无多余键                else 400 / accepted=false
11 x, y 是有限数且 |坐标| <= 2e6                else 400
12 channel 是整数且 1<=c<=20                    else 400
13 幂等: (path, request_id) 命中缓存            -> 返回首次响应
14 调用 Arena.{enter|measure|clear|exit}
15 回填 real_timestamp_ms 并写入幂等缓存
```

代码 6 do_POST 的校验顺序（短路求值，先便宜后昂贵）

两类拒绝的区别（附件 2 的语义）

400/404/413/415: 请求格式有问题，返回错误体，不消耗虚拟时间。
200 + accepted=false: 请求格式合法但语义上被拒绝（多余字段、arena_id 不匹配、robot_id 不匹配、重复 enter），同样不推进虚拟时间。这一区分很重要：机器狗程序若把两类混为一谈，会在参数写错时静默地空转。

6.2 幂等语义

```
1 rid = body["request_id"]; key = (path, rid)
2 if key in st.arena._rid_cache:
3     return self._send(200, st.arena._rid_cache[key]) # 直接回放首次响应
4 ...
5 resp["real_timestamp_ms"] = int(time.time() * 1000)
6 st.arena._rid_cache[key] = resp                    # 成功后落缓存
```

附件 2 规定同一 request_id 重发应返回首次结果（网络重试场景），内容不同则冲突。本实现的缓存键为 (path,request_id)，副作用只发生一次——即使机器狗因超时重发，也不会被重复计时或重复清除。

6.3 字段规范

指令	请求字段	响应字段	说明
/enter	arena_id, robot_id, request_id	accepted, virtual_time_s, max_virtual_ duration_s, max_real_ duration_s	建立会话，绑定 robot_id
/measure	position, channel	measure_result, svd_deg	svd_deg 仅在 direction 时出现
/clear	position, channel	clear_result	success / no_target_in_range
/exit	arena_id, robot_id, request_id	accepted, virtual_time_s, exit_reason	结束会话

指令	请求字段	响应字段	说明
全部	—	real_timestamp_ ms, virtual_time_s	每个成功响应都带

7 日志规范

每次动作写一行 JSON（JSON Lines，UTF-8，逐行 flush，便于中途中断后仍可分析）：

```
1 {"t": 274.0, "action": "measure", "pos": [650.0, 650.0], "channel": 7,  
2  "move_m": 400.0, "move_s": 80.0, "switch_s": 1.0, "result": "direction",  
3  "svd_deg": 63.42, "n_cleared": 2, "request_id": "m-000123"}
```

代码 7 日志行示例（measure 动作）

字段	类型	含义
t	float	动作 完成 后的虚拟时刻（s）
action	str	enter / measure / clear / exit
pos	[x,y]	动作执行位置（检测/清除为提交坐标, enter/exit 为当前位置）
channel	int	动作涉及频道（enter/exit 为当前频道）
move_m, move_s	float	本次动作隐含的移动距离与耗时
switch_s	float	频道切换耗时（0 或 1）
result	str	检测三态或清除结果
svd_deg	float?	仅 direction 时存在
n_cleared	int	该时刻累计已清除源数（便于画进度曲线）
request_id	str	与协议层一致，便于与 HTTP 抓包对齐

日志目录约定：正式测试写入 logs/formal/，文件名为 formal_seed< 种子 >.log；演练写入 logs/drill/，文件名为 drill_seed< 种子 >.log。

8 三种运行模式

模式	入口	用途与特点
内存模式	LocalClient (Arena(...))	大规模调参、60 组演练。无 HTTP 开销，单局 0.3~0.6 s
后台线程	run_server_thread (port, ...)	自动化脚本（run_http_tests.py）：在同进程内起服务线程，主线程用 HTTPClient 走真实 HTTP

模式	入口	用途与特点
阻塞服务	<code>serve(port, ...)</code> 或 <code>python simulator.py</code> <code>--port 2026 --seed</code> 42	正式测试/人工调试。命令行可指定 <code>--dir-mix</code> （定向源比例）、 <code>--n</code> （源个 数）、 <code>--log</code>

9 与附件协议的逐条对照

附件规定	本实现	验证方式
端口 2026、127.0.0.1	<code>serve(port=2026)</code>	实际联调通过
四条指令	完全一致	60 组演练 + 6 次正式测试
逐次等待响应、不并发	客户端同步调用	代码审查
Content-Type: application/json	接受带/不带 charset	单元测试
未声明字段 → <code>accepted=false</code>	字段白名单	构造多余字段请求验证
缺字段/类型错 → 400	校验流水线第 7/10~12 步	构造非法请求验证
同一 <code>request_id</code> → 幂等	<code>_rid_cache</code>	重发同一请求验证副作用只一次
移动 5 m/s、按直线	<code>_step</code>	日志 <code>move_m/move_s</code> 校验比值为 5
切换 1 s	<code>SWITCH_TIME</code>	日志 <code>switch_s</code> 逐条核对
检测 5 s	<code>MEASURE_TIME</code>	同上
清除命中 5 s / 未命中 3 s	<code>CLEAR_TIME_HIT/MISS</code>	同上
清除成功后该源不再被检测到	<code>src.cleared</code> 判定	<code>src is not None and not src.cleared</code>
$R_{\text{eff}} \in [1000, 1500]$ 每源不同	<code>make_case</code>	统计 60 局取值分布
$\pm 1^\circ$ 误差、同点一致	<code>_loc_error</code>	同点重测逐位相等
<code>near</code> : ≤ 5 m 且在覆盖角内	检测分支顺序	构造贴脸测试
频道 1~20 互异且不变	<code>rng.sample</code>	全局面检查
虚拟/真实时长上限	常量并随 <code>enter</code> 返回	响应字段核对

10 已知差异与局限

1. 不是官方模拟器。官方版本通过百度网盘分发，首次运行需联网注册并与竞赛服务器校验时间，正式测试额度绑定账号。本实现是按附件 1/2 的文字规范复现的**协议一致**模拟器，论文中的”正式测试”因此不是官方成绩。
2. 未实现的官方行为：真实的 `/enter` 可能返回剩余正式测试次数、可能有服务端随机种子下发等；本实现把 `arena_id` 固定为"default"，未做多局编排与限额管理。
3. 未模拟传输延迟与丢包。本地回环延迟约 1 ms，与”虚拟时间”无关；若真实环境有网络抖动，需要额外考虑真实时间预算。

4. 未模拟多机器人并发。题面只有一只机器狗，本实现一个服务进程只维护一份 Arena 状态，不支持并发会话。
5. 误差模型的无结构假设是保守的。真实测向误差常有空间相关性（同一建筑反射导致同向偏差），本实现刻意不引入这种结构，因此结论不会因“利用了现实中不存在的平滑性”而虚高。

10.1 如何接入官方模拟器

```
1 # 本地: cl = LocalClient(arena)
2 # 官方: 把端口指向 127.0.0.1:2026 (默认端口相同), robot_id 换成真实参赛队号
3 cl = HTTPClient(robot_id="<真实参赛队号>", port=2026, host="127.0.0.1")
4 brain = Brain(cl, params=P3)      # 策略代码完全不变
5 st = brain.run()
```

代码 8 切换回官方模拟器只需改一行

由于策略内核只通过客户端接口与外界交互（enter/exit/measure/clear 四个方法），且两个客户端的签名完全一致，切换不需要改动任何策略代码。